12. Risks and Technical Challenges

12.1 Introduction

The Local Service Problem Diagnosis and Matching System combines a web application, a text-classification model, and a database. These components must work together to help users identify the type of household or equipment problem they have and find suitable service providers.

Several technical challenges may arise during development. The main risks are incorrect classifications, limited training data, database errors, problems connecting application components, security issues, and limited development time. I will manage these risks by testing each component, keeping the project scope manageable, and checking the system regularly against its requirements.

12.2 Risk Assessment and Management Plan

Risk ID	Technical risk	Possible impact	Risk management plan
R1	Limited training data	The classification model may predict incorrect service categories because it has not learned enough examples.	Prepare a larger labeled dataset covering Plumbing, Electrical, and Appliance Repair. Test the model using separate training and testing data.
R2	Incorrect classification results	Users may receive unsuitable service recommendations if a problem is assigned to the wrong category.	Evaluate the model using accuracy, precision, recall, and F1-score. Test different descriptions and provide a fallback option when a prediction is uncertain.
R3	Integration problems	Flask, scikit-learn, and SQLite may not exchange data correctly, causing errors during request processing.	Test each component separately before testing the complete workflow. Check application logs and database records when errors occur.
R4	Database errors or data loss	Requests, predicted categories, or service records may not be saved correctly.	Use appropriate SQLite table structures, parameterized SQL queries, input validation, and regular database backups. Test whether saved records remain available after restarting the application.
R5	Incorrect provider matching	The system may recommend providers who do not offer the required service.	Store each provider's service category and match it against the predicted category. Test matching results using different service requests and provider records.
R6	Security and privacy problems	Unauthorized users may access restricted information or modify request records.	Implement authentication and role-based access where required. Validate user input, protect database operations, and test

			access restrictions before completion.
R7	Unfamiliar libraries or dependency problems	Installation errors or incompatible package versions may interrupt development.	Maintain a requirements.txt file, use a Python virtual environment, and document the installation and execution steps.
R8	Limited development time	Some planned features may remain incomplete before December 8, 2026.	Follow the development schedule, complete the main request-classification and storage workflow first, and add secondary features after the core system works.
R9	Performance problems	The application may respond slowly when processing multiple requests or running the classification model.	Measure response times under a defined workload and identify slow operations. Keep the model and database operations simple for the initial version.
R10	Scope changes and additional requirements	Adding unnecessary features may delay implementation and testing.	Keep the project focused on problem classification, provider matching, request storage, and request management. Record additional features as future improvements unless they are essential.

12.3 Risk Priorities

The most important risks are classification reliability, integration between the application components, and limited development time.

Classification reliability is important because the predicted service category affects provider matching. A model trained on a small sample dataset may work for simple examples but perform poorly on unfamiliar descriptions. I will evaluate it using held-out test data and review incorrect predictions before deciding whether it is suitable for the final system.

Integration is another major risk. The web interface must send the problem description to Flask, the classification model must return a prediction, and SQLite must save the result. I have already tested this basic workflow using the description "The kitchen sink pipe is leaking." The application returned Plumbing, displayed a confidence value of 0.51, and showed saved request ID 1. I will continue testing the database record and additional inputs to verify that the workflow works consistently.

Time is also a concern because the project must be completed by December 8, 2026. I will complete the essential features before adding optional improvements. If time becomes limited, I will prioritize reliable classification, database storage, provider matching, and request history over advanced features.

12.4 Contingency Plan

If the classification model does not produce acceptable results, I will review the training examples, text preprocessing, and model settings. I may simplify the categories or improve the dataset before considering a different classification method.

If database integration causes problems, I will test the database operations separately and use a small set of known records to identify the issue. If a feature cannot be completed within the available time, I will document its current status, explain the limitation, and keep the remaining implementation work within the project scope.

These measures will help me manage technical problems early and complete a working minimum viable product before the course deadline.

13. Development Schedule

13.1 Schedule Overview

The project will be developed in stages, beginning with architecture and design and ending with final testing, documentation, and presentation. The schedule covers October 27 through December 8, 2026.

The technical proof of concept has already been started. I have installed the required Python libraries and run a Flask application that accepts a problem description, predicts a service category, and saves a request in SQLite. The next stage is to expand this initial implementation into the core system.

13.2 Development Milestones

Milestone	Planned dates	Main activities	Expected deliverable
1. Architecture and Design	October 27–October 30, 2026	Review the project scope, functional and nonfunctional requirements, system architecture, database design, and user workflows.	Finalized architecture, database design, and updated project documentation.
2. Technical Proof of Concept	October 27–October 31, 2026	Verify the Flask application, text classification, SQLite storage, and basic browser interaction. Record test results and screenshots.	Working proof of concept, source code, database evidence, and Section 10 documentation.
3. Core Implementation	November 1–November 10, 2026	Improve problem classification, validate input, create database tables for users, providers, and requests, and implement request submission and storage.	Working core application with classification and request management functions.
4. Component Integration	November 11–November 16, 2026	Connect the web forms, Flask routes, classification model, database, and provider-matching logic. Verify that	Integrated application with classification, storage, and provider matching.

		information moves correctly between components.	
5. Minimum Viable Product (MVP)	November 17–November 20, 2026	Complete the essential user workflow, provider registration or management, request history, and basic status updates. Fix major integration issues.	A usable MVP demonstrating the main system requirements.
6. Testing and Evaluation	November 21–November 27, 2026	Conduct unit, integration, functional, performance, security, and acceptance tests as applicable. Evaluate the classification model using held-out data.	Test records, model evaluation results, identified defects, and completed test cases.
7. Refinement	November 28–December 1, 2026	Correct defects, improve input validation, refine classification where necessary, and address usability and performance issues. Repeat failed tests.	Improved system with documented test results and corrected defects.
8. Documentation	December 2–December 4, 2026	Complete the system description, implementation details, testing results, risk assessment, setup instructions, limitations, and supporting screenshots. Update the GitHub repository.	Complete project document, README, source code, and supporting evidence.
9. Final System and Presentation	December 5–December 8, 2026	Perform final end-to-end testing, verify repository contents, prepare the demonstration, and organize the final presentation and submission files.	Final working system, updated GitHub repository, final documentation, and presentation materials.

13.3 Weekly Work Plan

Period	Planned work	Completion target
October 27–October 31	Finish architecture updates and verify the proof of concept.	The basic classification and database workflow is documented and tested.
November 1–November 7	Develop core classification, input validation, and database operations.	Valid problem descriptions can be processed and saved.
November 8–November 14	Complete remaining core functions and begin connecting provider records to requests.	Classification and provider data can be connected through the application.
November 15–November 21	Finish provider matching, request history, and basic status management.	The MVP supports the main user workflow.
November 22–November 28	Perform planned tests, measure model performance, and record defects.	Testing evidence is available and major problems are identified.

November 29–December 5	Correct defects, complete documentation, and prepare the final demonstration.	The system and project documentation are ready for final review.
December 6–December 8	Run final checks, verify GitHub files, and complete submission preparations.	Final project deliverables are ready by December 8, 2026.

13.4 Progress Tracking and Schedule Management

I will track progress by comparing completed work with the planned milestones. At the end of each stage, I will review the expected deliverable and confirm that it has been completed before moving to the next stage.

The GitHub repository will be updated as development progresses. It will contain the application source code, dependency files, database setup instructions, documentation, and relevant testing evidence. I will keep the README updated so that another person can understand how to install and run the application. If a milestone takes longer than expected, I will prioritize the essential features and adjust lower-priority work. Classification, request storage, provider matching, and request history will receive priority because they support the main purpose of the system. Any unfinished feature or change to the schedule will be documented. The final deadline is December 8, 2026. By that date, I aim to deliver a working minimum viable product, test evidence showing how the system performs against its requirements, an updated GitHub repository, and complete project documentation.